

Aplicações Limitadas por Memória e por CPU

Eugênio Vitor Lopes dos Santos

DCA3703 – Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

Implemente dois programas paralelos em C com OpenMP: um limitado por memória, com somas simples em vetores, e outro limitado por CPU, com cálculos matemáticos intensivos. Paralelize com `#pragma omp parallel for` e meça o tempo de execução variando o número de threads. Analise quando o desempenho melhora, estabiliza ou piora. Reflita sobre como o multithreading de hardware pode ajudar programas memory-bound e atrapalhar programas compute-bound pela competição por recursos.

2 Fundamentação

Uma aplicação memory-bound transfere muitos dados e realiza poucas operações por byte. Neste caso, cada iteração lê dois valores e escreve um valor. O limite principal é a largura de banda da memória. A soma realiza uma operação de ponto flutuante para cerca de 24 bytes transferidos, sem considerar efeitos de cache.

Uma aplicação compute-bound realiza muitas operações antes de acessar novos dados. Neste trabalho, cada elemento executa 50 repetições de `sqrt` e `sin`. Essas funções exigem mais tempo de cálculo do que a leitura e a escrita de um único elemento. Não é correto atribuir um número exato de FLOPs a essas funções sem conhecer a implementação da biblioteca matemática. Por isso, a classificação é feita pelo comportamento do laço, e não por uma estimativa exata de FLOPs.

O multithreading de hardware pode ajudar uma aplicação memory-bound porque outra thread pode executar enquanto a primeira espera dados da memória. Esse ganho depende da latência e da largura de banda disponíveis. Quando a largura de banda está saturada, novas threads passam a competir pelo mesmo recurso. Em uma aplicação compute-bound, threads lógicas compartilham unidades de execução, caches e outros recursos do núcleo físico. Por isso, elas podem não produzir ganho e podem aumentar o tempo por causa da competição.

3 Implementação

Embora o enunciado use a expressão “dois programas”, esta implementação usa um único arquivo e um único executável. Ele contém dois testes independentes, executados e medidos separadamente. Essa escolha evita repetir o código de alocação, inicialização, medição e configuração de threads. Ela não altera os dois casos exigidos: o primeiro laço é memory-bound e o segundo é compute-bound.

O teste memory-bound aloca os vetores A, B e C e calcula $C[i] = A[i] + B[i]$. Cada thread escreve em uma posição diferente de C. O teste compute-bound aplica 50 repetições de `sqrt` e `sin` a cada elemento de V. O valor temporário `val` é privado de cada iteração. A alocação e o

preenchimento dos vetores ocorrem antes dos cronômetros. Cada tempo cobre somente o seu laço paralelo.

```
1 #define _POSIX_C_SOURCE 199309L
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <time.h>
6 #include <math.h>
7 #include <omp.h>
8
9 #define TOTAL_ELEMENTOS_PADRAO 50000000L
10
11 static double obter_tempo_segundos(void) {
12     struct timespec tempo_atual;
13     clock_gettime(CLOCK_MONOTONIC, &tempo_atual);
14     return (double)tempo_atual.tv_sec + (double)tempo_atual.tv_nsec / 1e9;
15 }
16
17 int main(int argc, char **argv) {
18     long total_elementos = argc > 1 ? atol(argv[1]) :
19         TOTAL_ELEMENTOS_PADRAO;
20     if (total_elementos <= 0) {
21         fprintf(stderr, "Uso: %s [total_elementos]\n", argv[0]);
22         return 1;
23     }
24
25     double *A = malloc((size_t)total_elementos * sizeof(*A));
26     double *B = malloc((size_t)total_elementos * sizeof(*B));
27     double *C = malloc((size_t)total_elementos * sizeof(*C));
28     double *V = malloc((size_t)total_elementos * sizeof(*V));
29
30     if (!A || !B || !C || !V) {
31         perror("malloc");
32         return 1;
33     }
34
35     for (long i = 0; i < total_elementos; ++i) {
36         A[i] = (double)i * 0.5;
37         B[i] = (double)i * 0.25;
38         V[i] = (double)i * 0.1;
39     }
40
41     int total_threads = omp_get_max_threads();
42
43     double tempo_inicio_memoria = obter_tempo_segundos();
44
45     #pragma omp parallel for
46     for (long i = 0; i < total_elementos; ++i) {
47         C[i] = A[i] + B[i];
48     }
```

```

47     }
48
49     double tempo_execucao_memoria = obter_tempo_segundos() -
        tempo_inicio_memoria;
50
51     double tempo_inicio_cpu = obter_tempo_segundos();
52
53     #pragma omp parallel for
54     for (long i = 0; i < total_elementos; ++i) {
55         double val = V[i];
56         for (int j = 0; j < 50; ++j) {
57             val = sqrt(val + j * 0.01) + sin(val);
58         }
59         V[i] = val;
60     }
61
62     double tempo_execucao_cpu = obter_tempo_segundos() - tempo_inicio_cpu;
63
64     printf("N=%ld threads=%d memory_bound=%.9f cpu_bound=%.9f\n",
65           total_elementos, total_threads, tempo_execucao_memoria,
66           tempo_execucao_cpu);
67
68     free(A); free(B); free(C); free(V);
69
70     return 0;
71 }

```

Listing 1: Testes limitado pela memória e limitado pela CPU

O arquivo é compilado com GCC, `-fopenmp` e `-lm`. O programa recebe o número de elementos como argumento e usa $N = 50.000.000$ como valor padrão. Os testes deste relatório usam $N = 5.000.000$. A medição usa `CLOCK_MONOTONIC`, que não é afetado por alterações no relógio do sistema.

```

1 gcc -std=c11 -Wall -Wextra -fopenmp tarefa04.c -o tarefa04 -lm
2
3 OMP_NUM_THREADS=1 ./tarefa04 5000000
4 OMP_NUM_THREADS=28 ./tarefa04 5000000

```

Listing 2: Compilação e execução dos testes

4 Resultados e Discussão

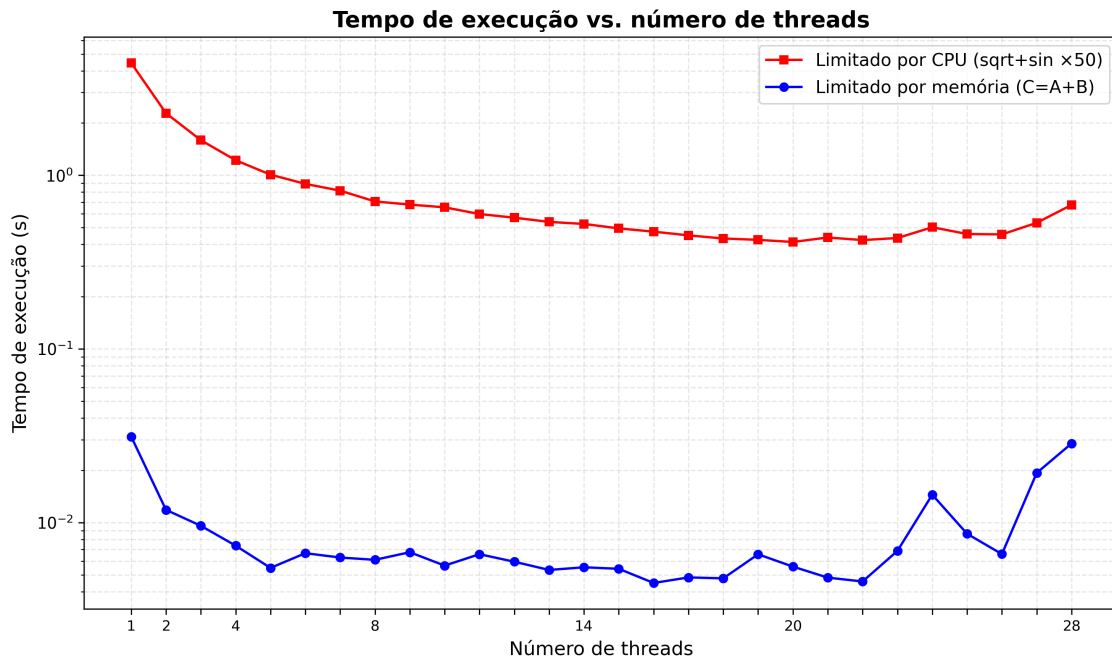


Figure 1: Tempo de execução em função do número de threads.

O programa memory-bound melhora quando o número de threads aumenta no início do experimento. Depois, o tempo apresenta pouca redução e oscila. Em alguns pontos, o tempo aumenta. Esse comportamento indica saturação ou disputa pela largura de banda da memória.

O programa compute-bound também melhora com o aumento de threads enquanto existem recursos de CPU disponíveis. Após esse ponto, o ganho diminui e pode desaparecer. Se threads lógicas compartilham o mesmo núcleo físico, a competição por unidades de cálculo pode causar estabilidade ou piora.

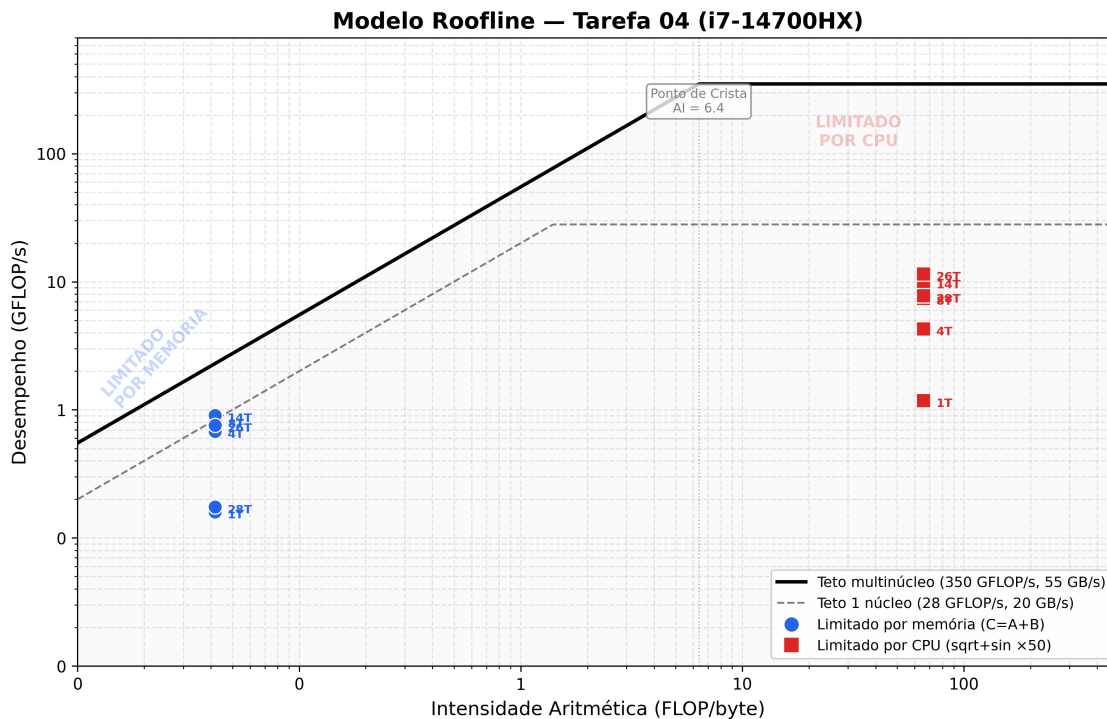


Figure 2: Modelo Roofline ajustado aos resultados com $N = 5.000.000$. Os tetos são estimativas de pico do sistema de referência.

O Roofline mostra onde o hardware travou o desempenho. O eixo X mede intensidade aritmética, quantas operações de ponto flutuante cada byte da memória rende. O eixo Y mostra GFLOP/s. A linha preta é o teto do sistema. À esquerda do ponto de crista, a memória é o gargalo. À direita, a CPU.

O kernel `memory_bound` faz uma adição por elemento.

```
C[i] = A[i] + B[i];
// 1 FLOP, 3 acessos à memória (ler A, ler B, escrever C) = 24 bytes
```

Intensidade: $1/24 \approx 0,042$ FLOP/byte. Bem à esquerda do ponto de crista. Com 16 threads os pontos sobem porque mais núcleos dividem a mesma largura de banda. A partir de 24 threads começam a cair. Threads extras competem pelo controlador de memória e reduzem o throughput efetivo.

O kernel `cpu_bound` roda 50 iterações pesadas por elemento.

```
for (j = 0; j < 50; ++j)
    val = sqrt(val + j*0.01) + sin(val);

// sqrt: ~8 FLOPs (libm, Newton-Raphson)
// sin: ~10 FLOPs (redução de argumento + polinômio minimax)
// Total: (1 MUL + 1 ADD + 8 + 10 + 1 ADD) × 50 1050 FLOPs
// Bytes: 2 acessos × 8 = 16 bytes/elemento
```

Intensidade: $\sim 1050/16 \approx 65,6$ FLOP/byte. Bem à direita do ponto de crista ($\sim 6,4$), onde a CPU define o limite. Com 20 threads sobe a ~ 120 GFLOP/s. Fica longe dos 350 GFLOP/s teóricos porque não usa vetorização AVX2/FMA e `sqrt/sin` rodam em software escalar.

`sqrt` e `sin` não são 1 FLOP cada. São funções de biblioteca com dezenas de operações internas, Newton-Raphson, polinômios minimax. A contagem de ~ 1050 FLOPs é uma estimativa razoável do custo real em software.

5 Conclusão

Os dois testes paralelos usam OpenMP e permitem comparar dois tipos de gargalo. Em uma aplicação memory-bound, o multithreading de hardware ajuda porque, enquanto uma thread aguarda dados da memória, outra thread pode executar. O ganho existe enquanto a largura de banda da memória não está saturada. Quando múltiplos núcleos lógicos compartilham o mesmo barramento de memória, threads adicionais competem pelo mesmo recurso e os ganhos se estabilizam ou diminuem. Em uma aplicação compute-bound, threads lógicas do mesmo núcleo físico compartilham unidades de execução, caches e etc. Por isso, o paralelismo pode não reduzir o tempo e até aumentá-lo: mais threads significam mais competição pelos mesmos recursos de cálculo, sem o benefício de esconder latência de memória.